

EDA using SQL on Python Notebook

Project Workflow: Load -> Clean -> EDA

1. Loading of Data

Import Mysql.connector

```
In [283... import mysql.connector
```

Create a function

```
In [286... def connect_to_db():                                # Create Function for calling database
    return mysql.connector.connect(
        host = " ",          # name of your host on MySQL Workbench
        user = " ",          # username on MySQL Workbench
        password = " ",      # password
        db = " "             # database you want to import
    )
connect_to_db().is_connected() # to check if we connected with the database
```

```
Out[286... True
```

Create an agent

```
In [323... # Creating an agent
db = connect_to_db()
cursor = db.cursor(dictionary=True) # The output will be in dictionary format
```

Example query

```
In [325... cursor.execute("select count(*) from electronicsdata") # query
row = cursor.fetchone() # fetch one query
list(row.values())[0] # convert it to list and fetch only the value
```

```
Out[325... 635
```

Describe the Data

```
In [327... import pandas as pd
cursor.execute("describe electronicsdata") # query
row = cursor.fetchall()
desc_data = pd.DataFrame(row)
desc_data
```

Out[327...

	Field	Type	Null	Key	Default	Extra
0	Sub Category	text	YES		None	
1	price	decimal(20,2)	YES		None	
2	MRP	decimal(50,2)	YES		None	
3	Discount_Type	text	YES		None	
4	Discount_Given	varchar(3)	YES		None	
5	Reviews_Count	int	YES		None	
6	Average_Rating	decimal(5,2)	YES		None	
7	Title	text	YES		None	
8	BrandName	varchar(50)	YES		None	
9	Feature	text	YES		None	

2. Clean

2.1 Check Distinct Values in Sub Category

```
In [329... cursor.execute("select distinct `Sub Category` from electronicsdata") # we are using back tick(`) not single quotes('')
row = cursor.fetchall()
distinct_values_sub_category = pd.DataFrame(row)
distinct_values_sub_category
```

Out [329...

Sub Category	
0	Batteries
1	Cameras & Camcorders
2	Desktop Computers & PCs
3	Gaming Desktop Computer & PCs
4	Gaming Laptops & Notebook Computers
5	Hard Drives, SSD & Storage
6	Headphones & Earbuds
7	Home Security Systems & Cameras
8	iPad & Tablets
9	Laptops & Notebook Computers
10	Mini PC Desktop Computer & PCs
11	Monitors
12	Musical Instruments
13	Phones & Two-way Radios
14	Projectors
15	Routers, Wifi Extenders & Modems
16	Smart Home & Safety
17	Smart Watches
18	Soundbars & Home Theater
19	Speakers & Voice Assistants
20	Streaming Devices
21	TVs
22	Video Games & Consoles

2.2 Remove '\$' from the `price` column values and replace vary price range of an item with average price

Update -> REMOVE '\$' and ',' from price VALUES

```
In [35]: cursor.execute("update electronicsdata set price = REPLACE(REPLACE(price, '$',''), ',','')")
db.commit()    # Commit the changes
```

Update -> Now Replace these old values with the new ones we have computed.

```
In [38]: cursor.execute("""update electronicsdata set price =
                        (substring_index(Replace(price, '-', ''), 'through', 1) +
                         substring_index(Replace(price, '-', ''), 'through', -1))/2
                        where price like '%through%'
                        """)
db.commit()    # Commit the changes
```

2.3 Create three columns from Discount column -> Discount_Type , MRP , Discount_Given

```
In [26]: cursor.execute("alter table electronicsdata add column Discount_Given varchar(3) after discount")
db.commit()
```

```
In [27]: # Insert values in Discount_Given Column
cursor.execute("""update electronicsdata
                set discount_given = case when discount like '%No Discount%' then 'No'
                else 'YES'
                end""")
db.commit()
```

```
In [50]: cursor.execute("alter table electronicsdata add column MRP decimal(50,2) after price")
db.commit()
```

```
In [52]: #Insert values in MRP Column
cursor.execute("""update electronicsdata
                set MRP = case when discount like '%No Discount%' then price
                when discount like '%After%' then round(price + regexp_replace(discount, '^[^0-9]', ''), 2)
                else round(price*1.27, 2)
                end
                """)
db.commit()
```

```
In [64]: # Rename of Discount column to Discount_Type
cursor.execute("alter table electronicsdata change column Discount Discount_Type text")
db.commit()
```

```
In [70]: #Update Data_Type with clear labels
cursor.execute("""update electronicsdata
                set Discount_Type = case when discount_type like '%No Discount%' then 'No Discount'
                when Discount_Type like '%Price Valid%' then 'Price Valid'
                when Discount_Type like '%After%' then 'Flat Discount Offers'
                else 'Special Discount'
                end
                """)

db.commit()
```

2.4 Drop Currency Column

```
In [83]: cursor.execute("alter table electronicsdata drop column Currency")
db.commit()
```

2.5 Extract Rating and Count of Rating from Rating column

```
In [92]: # Create a new column to extract the RATING
cursor.execute("alter table electronicsdata add Average_Rating decimal(5,2) after Rating")
db.commit()
```

```
In [98]: # Insert the rating in Average_Rating column from Rating column
cursor.execute("""update electronicsdata
                set Average_Rating = regexp_substr(rating, '[0-9]+(\\.[0-9]+)?')""")
db.commit()
```

```
In [102]: # Create a new column to extract the REVIEW_COUNT
cursor.execute("alter table electronicsdata add Reviews_Count int after Rating")
db.commit()
```

```
In [126]: # Insert the review_count in REVIEW_COUNT column from Rating column
cursor.execute("""update electronicsdata
                set Reviews_Count = regexp_substr(rating, '[0-9]+(?:= reviews)')""")
db.commit()
```

```
In [130]: # Now Drop the original Rating Column from the table
cursor.execute("alter table electronicsdata drop column Rating")
db.commit()
```

2.6 Extract BrandName from Title Column, first create new column

```
In [135... # Add new column
cursor.execute("alter table electronicsdata add BrandName varchar(50) after Title")
db.commit()
```

```
In [141... # Extract BrandName from Title
cursor.execute("""update electronicsdata
                  set BrandName = substring_index(Title, " ", 1)
                """)

db.commit()
```

2.7 In BrandName column there are 'ipad, aripods, imac' values, replace it to Apple

```
In [169... cursor.execute("""update electronicsdata
                  set BrandName = "Apple" where BrandName like '%mac%' or BrandName like '%ipad%' or
                  BrandName like '%Airpods%'
                """)

db.commit()
```

```
In [334... cursor.execute("select * from electronicsdata order by rand() limit 5")
row = cursor.fetchall()
view = pd.DataFrame(row)
view
```

Out [334...

	Sub Category	price	MRP	Discount_Type	Discount_Given	Reviews_Count	Average_Rating	Title	BrandName	Feature
0	Home Security Systems & Cameras	799.99	799.99	No Discount	No	41	3.80	Swann Home 16-Channel, 12-Camera Indoor/Outdoo...	Swann	Swannforce Red and Blue Controllable Flashing ...
1	TVs	479.99	609.59	Special Discount	YES	4440	4.50	Samsung 65" Class - CU7000D Series - 4K UHD LE...	Samsung	Crystal Processor 4K\nCrystal Display Slim D...
2	Cameras & Camcorders	349.99	444.49	Price Valid	YES	11	3.60	Insta360 GO 3 Action Camera Bundle	Insta360	Includes Pendant, Easy Clip, and Pivot Stand A...
3	Smart Home & Safety	219.99	219.99	No Discount	No	16	4.10	Blink - 3 Camera Security System - 2 Outdoor a...	Blink	Easy, DIY System 1080p HD Day and Night View...
4	Laptops & Notebook Computers	2699.99	3299.99	Flat Discount Offers	YES	31	3.20	MSI Raider GE78HX 17 Laptop - 13th Gen Intel C...	MSI	13th Gen Intel® Core™ i9-13950HX Processor N...

3. EDA

Univariate Analysis -> Numerical Columns

In [336...

```
# Check Null Values -> PRICE
cursor.execute("select count(*) as 'NULL Values' from electronicsdata where price is null")
row = cursor.fetchall()
null_values = pd.DataFrame(row)
null_values
```

Out[336... **NULL Values**

0	0
----------	---

```
In [38]: # Before converting data type remove this column
cursor.execute("delete from electronicsdata where `Sub Category` = 'Home Security Systems & Cameras' and BrandName = 'Lorex'")
db.commit()
```

```
In [40]: # Now change the type for PRICE Column
cursor.execute("alter table electronicsdata modify column price decimal(20,2)")
db.commit()
```

```
In [338... # Find mean, min, max and std -> PRICE
cursor.execute("""select min(price) as Min, max(price) as Max, avg(price) as Mean, std(price) as STD
                  from electronicsdata""")
row = cursor.fetchall()
stats_values = pd.DataFrame(row)
stats_values
```

	Min	Max	Mean	STD
0	14.99	7999.99	940.709087	944.184835

```
In [340... # Histogram
cursor.execute("""select buckets, count(*) from
                  (select price, case
                     when price between 0 and 500 then '0-0.5K'
                     when price between 501 and 1500 then '0.5K-1.5K'
                     when price between 1501 and 3000 then '1.5K-3K'
                     when price between 3001 and 6000 then '3K-6K'
                     else '>6K'
                  end as 'buckets'
                  from electronicsdata) k
                  group by buckets
                  """)
row = cursor.fetchall()
histogram = pd.DataFrame(row)
histogram
```


Out[340...

	buckets	count(*)
0	0-0.5K	271
1	0.5K-1.5K	246
2	1.5K-3K	100
3	3K-6K	17
4	>6K	1

In [342...

```
# Check NULL Value -> MRP
cursor.execute("select count(*) as 'NULL Values' from electronicsdata where MRP is null")
row = cursor.fetchall()
null_values_mrp = pd.DataFrame(row)
null_values_mrp
```

Out[342...

	NULL Values
0	0

In [344...

```
# Find mean, min, max and std -> MRP
cursor.execute("""select min(MRP) as Min, max(MRP) as Max, avg(MRP) as Mean, std(MRP) as STD
                  from electronicsdata""")
row = cursor.fetchall()
stats_values_mrp = pd.DataFrame(row)
stats_values_mrp
```

Out[344...

	Min	Max	Mean	STD
0	14.99	170000799.99	683545.086252	9.914235e+06

In [346...

```
# Check NULL Value -> Reviews_Count
cursor.execute("select count(*) as 'NULL Values' from electronicsdata where Reviews_Count is null")
row = cursor.fetchall()
null_values_reviews_count = pd.DataFrame(row)
null_values_reviews_count
```

Out[346...

	NULL Values
0	37

```
In [348... # Find mean, min, max and std -> Reviews_Count
cursor.execute("""select min(Reviews_Count) as Min, max(Reviews_Count) as Max, avg(Reviews_Count) as Mean,
                  std(Reviews_Count) as STD from electronicsdata""")
row = cursor.fetchall()
stats_values_reviews_count = pd.DataFrame(row)
stats_values_reviews_count
```

```
Out[348...      Min    Max    Mean    STD
0      1  10701  500.6706  1035.698759
```

```
In [350... # Check NULL Value -> Average_Rating
cursor.execute("select count(*) as 'NULL Values' from electronicsdata where Average_Rating is null")
row = cursor.fetchall()
null_values_avg_rating = pd.DataFrame(row)
null_values_avg_rating
```

```
Out[350...      NULL Values
0              37
```

```
In [352... # Find mean, min, max and std -> Average_Rating
cursor.execute("""select min(Average_Rating) as Min, max(Average_Rating) as Max, avg(Average_Rating) as Mean,
                  std(Average_Rating) as STD from electronicsdata""")
row = cursor.fetchall()
stats_values_average_rating = pd.DataFrame(row)
stats_values_average_rating
```

```
Out[352...      Min    Max    Mean    STD
0    1.00    5.00  4.333946  0.47422
```

Categorical Column

```
In [354... # Check NULL Value -> `Sub Category`
cursor.execute("select count(*) as 'Null Values' from electronicsdata where `Sub Category` is null")
row = cursor.fetchall()
null_values_sub_category = pd.DataFrame(row)
null_values_sub_category
```

Out[354... **Null Values**

0 0

```
In [356... # Pie Chart
cursor.execute("select `Sub Category`, count(`Sub Category`) as 'Counts' from electronicsdata group by `Sub Category`")
sub_category_pie = cursor.fetchall()
sub_category_pie
```

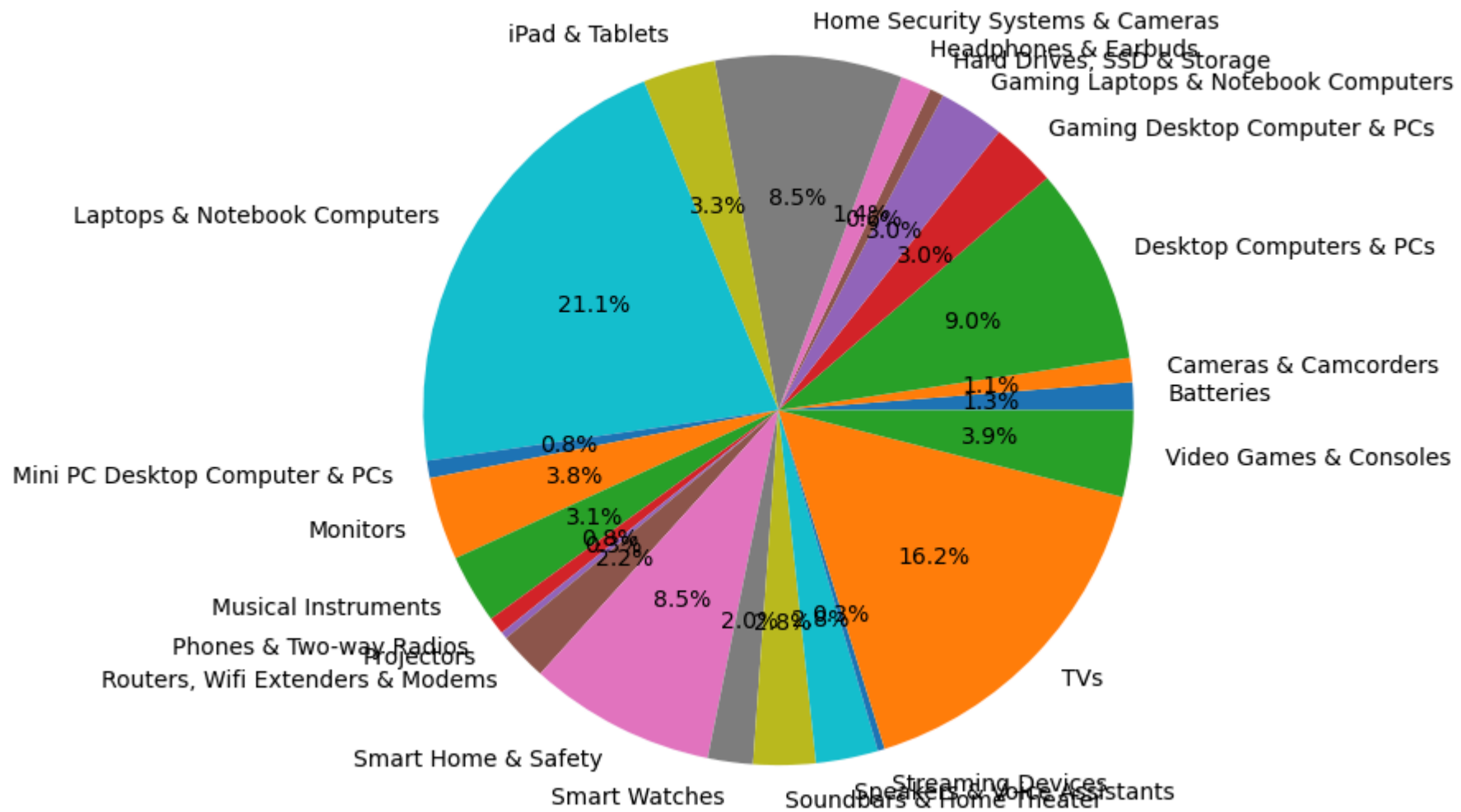
```
Out[356... [{'Sub Category': 'Batteries', 'Counts': 8},
{'Sub Category': 'Cameras & Camcorders', 'Counts': 7},
{'Sub Category': 'Desktop Computers & PCs', 'Counts': 57},
{'Sub Category': 'Gaming Desktop Computer & PCs', 'Counts': 19},
{'Sub Category': 'Gaming Laptops & Notebook Computers', 'Counts': 19},
{'Sub Category': 'Hard Drives, SSD & Storage', 'Counts': 4},
{'Sub Category': 'Headphones & Earbuds', 'Counts': 9},
{'Sub Category': 'Home Security Systems & Cameras', 'Counts': 54},
{'Sub Category': 'iPad & Tablets', 'Counts': 21},
{'Sub Category': 'Laptops & Notebook Computers', 'Counts': 134},
{'Sub Category': 'Mini PC Desktop Computer & PCs', 'Counts': 5},
{'Sub Category': 'Monitors', 'Counts': 24},
{'Sub Category': 'Musical Instruments', 'Counts': 20},
{'Sub Category': 'Phones & Two-way Radios', 'Counts': 5},
{'Sub Category': 'Projectors', 'Counts': 2},
{'Sub Category': 'Routers, Wifi Extenders & Modems', 'Counts': 14},
{'Sub Category': 'Smart Home & Safety', 'Counts': 54},
{'Sub Category': 'Smart Watches', 'Counts': 13},
{'Sub Category': 'Soundbars & Home Theater', 'Counts': 18},
{'Sub Category': 'Speakers & Voice Assistants', 'Counts': 18},
{'Sub Category': 'Streaming Devices', 'Counts': 2},
{'Sub Category': 'TVs', 'Counts': 103},
{'Sub Category': 'Video Games & Consoles', 'Counts': 25}]
```

```
In [358... from matplotlib import pyplot as plt

labels = [row['Sub Category'] for row in sub_category_pie]
counts = [row['Counts'] for row in sub_category_pie]

fig = plt.figure(figsize=(10, 7))
plt.pie(counts, labels=labels, autopct='%1.1f%%')
plt.title("Distribution of Sub Categories")
plt.show()
```

Distribution of Sub Categories



Bivariate Analysis -> Numerical Columns

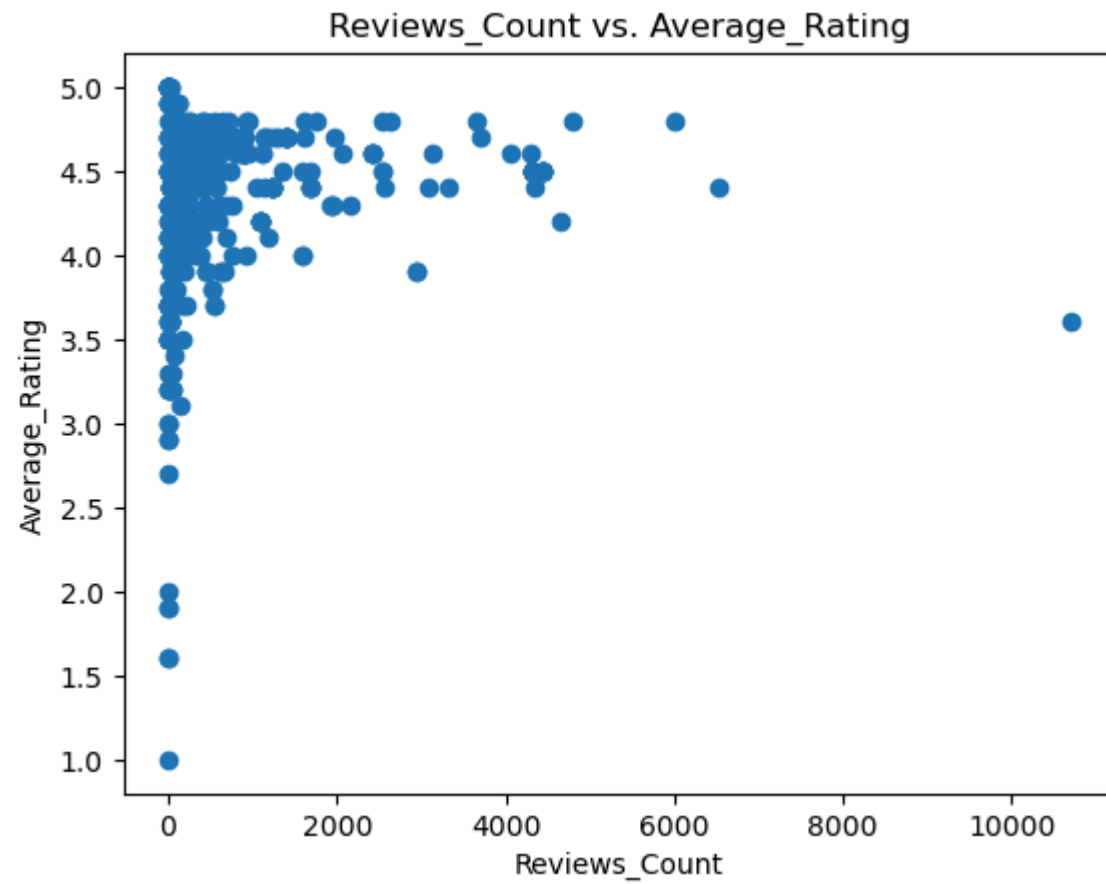
```
In [ ]: # Scatter Plot -> Reviews_Count and Average_Rating
        cursor.execute("select Reviews_Count, Average_Rating from electronicsdata")
        scatter_rev_count_avg_rating = cursor.fetchall()
        scatter_rev_count_avg_rating
```

```
In [362]: from matplotlib import pyplot as plt

          x = [row['Reviews_Count'] for row in scatter_rev_count_avg_rating]
```

```
y = [row['Average_Rating'] for row in scatter_rev_count_avg_rating]
```

```
plt.scatter(x, y)  
plt.title("Reviews_Count vs. Average_Rating")  
plt.xlabel("Reviews_Count")  
plt.ylabel("Average_Rating")  
plt.show()
```



```
In [364... # Find COVARIANCE
cursor.execute("""
    select
    round(sum((average_rating - (select avg(average_rating) from electronicsdata)) *
    (reviews_count - (select avg(reviews_count) from electronicsdata)))/
    (count(*) -1),2) as COVARIANCE
    from electronicsdata
    """)
row = cursor.fetchall()
covariance = pd.DataFrame(row)
covariance
```

```
Out[364... COVARIANCE
0          55.47
```

```
In [366... # Find CORELATION
cursor.execute("""
    select
    round((round(sum((average_rating - (select avg(average_rating) from electronicsdata)) *
    (reviews_count - (select avg(reviews_count) from electronicsdata)))/
    (count(*) -1),2))/
    (std(average_rating)*std(reviews_count)),3)
    as CORELATION
    from electronicsdata
    """)
row = cursor.fetchall()
corelation = pd.DataFrame(row)
corelation
```

```
Out[366... CORELATION
0          0.113
```

```
In [368... # Find SLOPE
cursor.execute("""
    select
    round(sum((average_rating - (select avg(average_rating) from electronicsdata)) *
    (reviews_count - (select avg(reviews_count) from electronicsdata)))/
    (sum(power(average_rating - (select avg(average_rating) from electronicsdata), 2))),2)
    as SLOPE
    from electronicsdata
    """)
row = cursor.fetchall()
```

```
slope = pd.DataFrame(row)
slope
```

```
Out[368...  SLOPE
0      261.52
```

Bivariate Analysis -> Categorical Columns

```
In [376... # Two rows are in the dataset have price first then brandname in BrandName Column, we'll correct that.
cursor.execute("update electronicsdata set BrandName = 'Ninetendo' where BrandName like '%$%'")
```

```
In [386... # Now we'll group by BrandName
cursor.execute("""select BrandName, count(distinct(`Sub Category`)) as Sectors from electronicsdata
                group by BrandName""")
row = cursor.fetchall()
group_by_1 = pd.DataFrame(row)
group_by_1
```

```
Out[386...  BrandName  Sectors
0      Acer      4
1      ADT       2
2    Allstate    1
3     Apple     8
4  Arcade1Up     1
...      ...      ...
85  Washburn     1
86     Wyze      1
87     Xbox      1
88     Yale      1
89   Yamaha      1
```

90 rows x 2 columns

Bivariate Analysis -> Numerical and Categorical Columns

```
In [397... cursor.execute(""" select BrandName,
                        round(min(price),2) as MIN,
                        round(max(price),2) as MAX,
                        round(avg(price),2) as MEAN,
                        round(std(price),2) as STD
                        from electronicsdata
                        group by BrandName
                        """)

row = cursor.fetchall()
group_by_2 = pd.DataFrame(row)
group_by_2
```

Out [397...

	BrandName	MIN	MAX	MEAN	STD
0	Duracell	14.99	20.99	19.32	2.13
1	Jackery	374.99	1324.99	849.99	475.00
2	GoPro	239.97	289.99	264.98	25.01
3	Insta360	349.99	399.99	374.99	25.00
4	DJI	429.99	839.99	634.99	205.00
...	...	...	...	...	...
85	Meta	599.99	599.99	599.99	0.00
86	My	79.99	79.99	79.99	0.00
87	PlayStation	629.99	629.99	629.99	0.00
88	Arcade1Up	199.99	949.99	533.32	311.80
89	Backbone	49.97	49.97	49.97	0.00

90 rows x 5 columns

Multivariate Analysis -> Mutliple Columns

```
In [400... cursor.execute("""select `Sub Category`,
                        round(sum((price - (select avg(price) from electronicsdata)) *
                        (mrp - (select avg(mrp) from electronicsdata))))/
                        (count(*) -1),2) as MULTI_COVARIANCE
                        from electronicsdata
                        group by `Sub Category`
                        """)
```



```
row = cursor.fetchall()
multi_var = pd.DataFrame(row)
multi_var
```

Out[400...

	Sub Category	MULTI_COVARIANCE
0	Batteries	557591975.39
1	Cameras & Camcorders	362633561.36
2	Desktop Computers & PCs	-252439592.35
3	Gaming Desktop Computer & PCs	-602774422.41
4	Gaming Laptops & Notebook Computers	2215234733.33
5	Hard Drives, SSD & Storage	777065064.54
6	Headphones & Earbuds	583111997.91
7	Home Security Systems & Cameras	287605428.78
8	iPad & Tablets	282846166.84
9	Laptops & Notebook Computers	1163769365.23
10	Mini PC Desktop Computer & PCs	-619900665.93
11	Monitors	455034901.08
12	Musical Instruments	-507016290.99
13	Phones & Two-way Radios	726775160.37
14	Projectors	-2460653614.47
15	Routers, Wifi Extenders & Modems	301939649.87
16	Smart Home & Safety	367237497.11
17	Smart Watches	422415134.67
18	Soundbars & Home Theater	390066028.01
19	Speakers & Voice Assistants	492321503.72
20	Streaming Devices	1145750270.59
21	TVs	-366357733.60
22	Video Games & Consoles	475163546.87

```
In [402... cursor.execute("select * from electronicsdata")
row = cursor.fetchall()
view_new_dataset = pd.DataFrame(row)
view_new_dataset
```

	Sub Category	price	MRP	Discount_Type	Discount_Given	Reviews_Count	Average_Rating	Title	BrandName	Feature
0	Batteries	20.99	20.99	No Discount	No	NaN	None	Duracell Coppertop Alkaline AA Batteries, 40- c...	Duracell	POWER BOOST™ Ingredients Guaranteed For 12 Y...
1	Batteries	20.99	20.99	No Discount	No	NaN	None	Duracell Coppertop Alkaline AAA Batteries, 40-...	Duracell	POWER BOOST™ Ingredients Guaranteed For 12 Y...
2	Batteries	20.99	20.99	No Discount	No	NaN	None	Duracell 9V Alkaline Batteries, 8- count	Duracell	Long-Lasting Batteries Designed for Dependabil...
3	Batteries	18.99	18.99	No Discount	No	NaN	None	Duracell C Alkaline Batteries, 14- count	Duracell	Long-Lasting Batteries Designed for Dependabil...
4	Batteries	14.99	14.99	No Discount	No	125.0	4.80	Duracell 2032 Lithium Coin Batteries, 12- count	Duracell	Our Longest Lasting Lithium Coin Battery Ever ...
...	...	...	...	...	...	...	...	...	...	...
630	Video Games & Consoles	949.99	949.99	No Discount	No	27.0	4.00	Arcade1Up Infinity Game Table 32", \$100 Game C...	Arcade1Up	Tactile Feedback to add Levels of Immersion ...
631	Video Games & Consoles	449.99	449.99	No Discount	No	14.0	3.90	Arcade1Up Infinity Game Board 18.5", \$25 Game ...	Arcade1Up	50 Board Games and Activities Included 18.5-...
632	Video Games & Consoles	199.99	199.99	No Discount	No	12.0	4.70	Arcade1Up NBA Jam Partycade 3 Games in 1	Arcade1Up	3 Games Included 17- inch LCD Color Screen ...
633	Video Games &	49.97	49.97	No Discount	No	180.0	3.90	Backbone One	Backbone	The Backbone App is Your

	Sub Category	price	MRP	Discount_Type	Discount_Given	Reviews_Count	Average_Rating	Title	BrandName	Feature
	Consoles							(Lightning) - PlayStation Edition...		Gaming & Streaming Hu...
634	Video Games & Consoles	599.99	599.99	No Discount	No	50.0	4.70	Xbox Series X Diablo IV Bundle with Additional...	Xbox	Xbox Series X Console Diablo® IV for Xbox On...

635 rows × 10 columns

FINISH.....